

BUỔI 7: Cấu trúc mảng một chiều

I. Lý thuyết

1. Định nghĩa

- Mảng là một tập hợp gồm nhiều phần tử có cùng kiểu dữ liệu và chung một tên, được phân biệt với nhau bởi chỉ số mảng.
- Ví dụ: tập hợp các số nguyên tạo thành một mảng nguyên, tập hợp các ký tự tạo thành một mảng ký tự.

Mảng số nguyên:

4	5	2	-1	6	3
---	---	---	----	---	---

Mảng ký tự:

A	C	W	B	Q	N
---	---	---	---	---	---

- Mảng có mảng 1 chiều, 2 chiều,... và mỗi kiểu dữ liệu thì có một kiểu mảng tương ứng (mảng nguyên, mảng thực, mảng ký tự,...).

2. Khai báo

Kieu_du_lieu *ten_mang*[*kich_thuoc*]

Ví dụ: `int a[10];` // Khai báo mảng nguyên a, có tối đa 10 phần tử

`float b[30];` // Khai báo mảng số thực b có tối đa 30 phần tử

Có thể vừa khai báo, vừa gán giá trị ban đầu như sau :

Kieu_du_lieu *ten_mang*[*n*] = { *gt1*, *gt2*, ..., *gtk* } (*k* ≤ *n*)

Ví dụ : `int a[5] = {1, 2, 3, 4, 5}`

`int a[] = {1, 2, 3}`

3. Truy cập vào phần tử mảng

Sau khi mảng được khai báo, mỗi phần tử trong mảng đều có chỉ số để tham chiếu. Chỉ số bắt đầu từ 0 đến *n*-1 (với *n* là kích thước mảng).

Cách truy cập :

Ten_mang [*chi_so_1*][*chi_so_2*]...[*chi_so_n*]

Ví dụ: `a[0]`, `a[1]`, ..., `a[i]`, ..., `a[n-1]`

Lấy địa chỉ phần tử trong mảng: `&a[i]`

4. Chú ý

- Các phần tử trong mảng nằm ở các ô nhớ liên tiếp nhau trong bộ nhớ
- Các phần tử mảng được truy cập bằng cách sử dụng một chỉ số nguyên (có thể là số thực và lấy phần nguyên)
- Nếu truy xuất tới các chỉ số nằm ngoài phạm vi mảng sẽ trả về giá trị không chính xác.

II. Ví dụ

Ví dụ 1: Viết chương trình nhập vào một dãy số nguyên có *n* phần tử. In dãy ra màn hình theo chiều xuôi và chiều đảo ngược.

Hướng dẫn:

- Input: *n*, mảng nguyên *a*

- Output: in mảng theo chiều xuôi và chiều đảo ngược
- Xác định biến và kiểu dữ liệu của biến:
 - o Biến nguyên n: chứa số phần tử của mảng
 - o Mảng nguyên A, có max phần tử
 - o Biến nguyên i, chỉ số trên mảng
- Thuật toán:
 - o B1: Nhập n và mảng nguyên A
 - o B2: Duyệt từ đầu đến cuối mảng (dùng for) để in từng phần tử của A
 - o B3: Duyệt từ cuối về đầu mảng (dùng for) để in từng phần tử của A
 - o B4: Kết thúc.
- Chương trình

```
#include <stdio.h>
#include <stdlib.h>
#define Max 100
int main()
{
    int A[Max], n;
    int i;
    printf("Nhap so phan tu cua mang:"); scanf("%d", &n);
    for (i=0; i<n; i++)
    {
        printf("A[%d]=", i);
        scanf("%d", &A[i]);
    }
    printf("\n Mang in theo chieu xuoi:");
    for (i=0; i<n; i++)
        printf("%5d", A[i]);
    printf("\n Mang in theo chieu nguoc:");
    for (i=n-1; i>=0; i--)
        printf("%5d", A[i]);

    return 0;
}
```

Sửa đổi chương trình trên với yêu cầu $n \geq 5$, nếu n nhỏ hơn 5 yêu cầu nhập lại (sử dụng vòng lặp).

Ví dụ 2: Nhập vào một mảng số nguyên. Tìm tổng của tất cả các số trong mảng đó, tìm tổng các số nguyên dương, tổng của các số nguyên âm. Tìm trung bình cộng của cả mảng, trung bình cộng của các số nguyên dương, trung bình cộng của các số nguyên âm.

Hướng dẫn:

+ Input: Mảng nguyên A

+ Output: Tổng, tổng dương, tổng âm, trung bình cộng, trung bình cộng số dương, trung bình cộng số âm.

+ Thuật toán:

- B1: Nhập số phần tử mảng, nhập mảng
- B2: Lặp từ đầu đến cuối mảng:
 - o $T = T + a[i]$
 - o Nếu $a[i] > 0$: $TD = TD + a[i]$; $d1 = d1 + 1$; // TD là tổng các số dương, d1 là số phần tử dương
 - o Nếu $a[i] < 0$: $TA = TA + a[i]$; $d2 = d2 + 1$; // TA là tổng các số âm, d2 là số phần tử âm

- B3 : $TBC = T/n$; $TBD = TD/d1$; $TBA = TA/d2$; // TBC là trung bình cộng, TBD là trung bình cộng các số dương, TBA là trung bình cộng các số âm
- B4 : In các giá trị: TA, TD, TA, TBC, TBD, TBA

+ Chương trình:

```
int main()
{
    int A[Max], n, i, T, TA, TD, d1, d2
    float TBC, TBD, TBA;
    printf("Nhap so phan tu cua mang:"); scanf("%d", &n);

    for (i=0; i<n; i++)
    {
        printf("A[%d]=", i);
        scanf("%d", &A[i]);
    }
    for (i=0; i<n; i++)
    {
        T=T+A[i]; //Tổng các phần tử
        if (A[i]>0)
        {
            TD=TD+A[i]; //Tổng các số dương
            d1=d1+1;
        }
        if (A[i]<0)
        {
            TA=TA+A[i]; //Tổng các số âm
            d2=d2+1;
        }
    }
    TBC=T/n;
    TBD=TD/d1;
    TBA=TA/d2;
    printf("\nTong cac phan tu trong mang = %d", T);
    printf("\n Tong cac so nguyen duong la %d", TD);
    printf("\n Tong cac so nguyen am la %d", TA);
    printf("\n Trung binh cong la %.2f", TBC);
    printf("\n Trung binh cong cac so duong la %.2f", TBD);
    printf("\n Trung binh cong cac so am la %.2f", TBA);
    return 0;
}
```

+ Kiểm thử chương trình với các bộ dữ liệu sau. Đánh giá và sửa lại chương trình cho chính xác.

STT	Input	Output Expected
1	n=6 {3,2,-1,-6,4,5}	Tổng = 7; Tổng dương = 14, tổng âm = -7, TBC=1.17, TBD= 3.5, TBA =- 3.5
2	n=4 {1,2,3,4}	Tổng = 10; Tổng dương = 10, tổng âm = 0, TBC=2.5, TBD= 2.5, TBA = 0

Ví dụ 3: Viết chương trình nhập vào dãy số nguyên. Tìm và in ra phần tử lớn nhất, bé nhất trong dãy kèm vị trí. Ví dụ: cho dãy {5, 2, -1, 4, -3, -1, 5}. Kết quả in ra là:

Số lớn nhất là 5, xuất hiện tại vị trí 0, 6

Số nhỏ nhất là -1 xuất hiện tại vị trí 2, 5

Hướng dẫn:

+ Thuật toán:

- B1: Nhập n và dãy số nguyên.
- B2: max = a[0], min = a[0]
- B3: Lặp từ phần tử thứ 2 đến cuối dãy
 - o Nếu max < a[i]: max = a[i], v1 = i;
 - o Nếu min > a[i]: min = a[i], v2 = i;
- B4: In max và vị trí v1
- B5: In min và in vị trí v2
- B6: Kết thúc

+ Chương trình:

```
int main()
{
    int A[100], i, max, min, n, v1, v2;
    printf("Nhập số phần tử của mảng:"); scanf("%d", &n);
    for (i=0; i<n; i++)
    {
        printf("A[%d]=", i);
        scanf("%d", &A[i]);
    }
    max=A[0]; v1=0;
    min=A[0]; v2=0;
    for (i=0; i<n; i++)
    {
        if (max<A[i]) {max=A[i], v1=i};
        if (min>A[i]) {min= A[i], v2=i};
    }
    printf("\nPhần tử lớn nhất là %d xuất hiện tại vị trí: %d", max, v1);
    printf("\nPhần tử nhỏ nhất là %d xuất hiện tại vị trí: %d", min, v2);
    return 0;
}
```

+ Kiểm thử chương trình với các bộ sau. Nhận xét kết quả và sửa lại chương trình cho chính xác.

STT	Input	Output Expected
1	n=6 {3,5,-1,-6,4,2}	Số lớn nhất là 5 xuất hiện tại vị trí 2 Số nhỏ nhất là -6, xuất hiện tại vị trí 3
2	n=6 {-1,2,4,-1,3,4}	Số lớn nhất là 4, xuất hiện tại vị trí 2, 5 Số nhỏ nhất là -1 xuất hiện tại vị trí 0, 3

III. Bài tập (yêu cầu sinh viên tự code)

Câu 23: Viết chương trình nhập vào dãy số thực, cho biết số x nào đó xuất hiện bao nhiêu lần trong dãy (x là số nhập vào) cùng với vị trí xuất hiện tương ứng.

Câu 24: Viết chương trình nhập vào dãy số thực, sắp xếp dãy theo chiều tăng /giảm dần

Câu 25: Viết chương trình nhập vào dãy số, thay thế các phần tử có giá trị âm trong mảng bằng giá trị 0.

Câu 26: Viết chương trình nhập vào một dãy số nguyên, hãy sắp xếp dãy theo nguyên tắc: số âm đầu dãy, số dương cuối dãy.

Ví dụ: Cho dãy: {2, -1, 3, 5, -4, -6}, kết quả in ra là: {-6, -1, -4, 5, 3, 2}

Câu 27: Viết chương trình nhập vào dãy số nguyên, đếm tần suất xuất hiện của các số trong dãy.

Ví dụ: cho dãy: {1, 3, 4, 1, 3, 1}, kết quả in ra là:

Số 1 xuất hiện 3 lần

Số 3 xuất hiện 2 lần

Số 4 xuất hiện 1 lần

Câu 27*: Cho hai dãy đã sắp xếp, hãy trộn hai dãy để được dãy thứ 3 mà vẫn đảm bảo tính sắp xếp của dãy thứ 3.

Ví dụ: cho dãy 1 2 5 8 và 0 4 6 10 11. Kết quả trộn hai dãy ta được dãy sau: 0 1 2 4 5 6 8 10 11